

Structural Auditing of Decision Claims: A Deterministic Offline Method and Its Reproducible Evidence Chain

Second-Perspective Language 2026.1 and the Global Cognitive Audit Engine (GCAE v2.0.0)

Author: Zhichen Ji

Affiliation: NOHN AI TECHNOLOGY PTE. LTD., Singapore

Contact: ai@nohnlins.com · ORCID 0009-0006-9578-5657

Date: 2026-10-09 · Version 1.0 — public technical report

Abstract

Machine-generated decision claims — the outputs of large language models (LLMs) in high-stakes contexts — are narratives: fluent, plausible, and structurally incomplete. Current evaluation of such outputs concentrates on content truthfulness. We address a different, orthogonal dimension: **structural completeness and evidentiary properties**. This paper specifies a deterministic, offline, model-agnostic method for auditing the structure of decision claims, comprising (i) a formal decision-structure language with a machine-checkable grammar and a validator that rejects conclusion-bearing, recommendation-bearing, ranking, and optimisation text by construction (19 diagnostic codes); (ii) a five-operator audit pipeline (narrative stripping, implicit-assumption surfacing, fragility assessment, causal-chain synchronisation, state anchoring) with an explicit severity model in which only structural absence — never quantitative opinion — may block an audit; and (iii) a tamper-evident, reproducible evidence chain whose integrity any third party can re-verify offline. We define a structural taxonomy of decision claims — *anchored / floating / opaque / hollow* — that is independent of factual truth and measurable by a fixed procedure. We report the results of a shipped, zero-dependency verification suite (11 checks: 9 pass, 2 disclosed limitations) and publish the resulting chain root (f5713f49...) as a cross-machine reproducibility reference. The method does not evaluate whether a claim is true; it records, in a form that survives adversarial scrutiny, **whether the claim declares the assumptions it depends on and the responses it commits to when those assumptions fail** — the evidence layer required by record-keeping, transparency, and human-oversight obligations of high-risk AI governance regimes.

摘要 —— 大模型在高风险场景下输出的“决策主张”本质是叙事：流畅、可信、但结构不完整。本文不评估内容真伪（那一维已有大量工作），而是给出结构完整性与证据属性这一正交维度的确定性、离线、与模型无关的审计方法：① 一门可机器校验的决策结构语言（19条诊断码，按构造拒绝结论/建议/排序/优化类表述）；② 五算子审计流水线（叙事剥离、内隐假设透视、脆弱性锁、因果链同步、状态锚定），其严重度模型规定：只有结构性缺失可以阻断，量化意见不能；③ 可防篡改、可复现的证据链，任何第三方可离线重算。另给出与事实真伪无关、可按固定程序测量的结构分类法（锚定/浮置/不透明/空洞）。随附零依赖验证套件实测结果（11项：9通过、2项如实披露）与跨机复现链根。方法不判断主张真假，只记录“该主张是否声明了所依赖的假设、以及假设失效时所承诺的分支响应”——这正是高风险AI治理中记录保持、透明度与人类监督义务所需的证据层。

1. Introduction

An LLM asked whether an eighteen-month, multi-million-unit transformation plan is feasible will typically answer with a conclusion, supporting reasons, and industry examples. The answer is fluent, and it may well be correct. It will also, in the overwhelming majority of cases, omit three things:

1. **the assumptions** on which the conclusion silently depends (data quality, organisational capacity, regulatory constraints);
2. **the failure branches** — what the decision becomes when any of those assumptions fails;
3. **any reproducible evidence** linking this output to this input at a later time, to a different party.

None of these omissions is a factual error. All three are structural. We call the class of omissions **structural hallucination** — confident assertions whose supporting structure is undeclared — and treat it as a measurable property distinct from content

truthfulness [5, 6].

This distinction matters for governance. High-risk AI regimes — the EU AI Act's requirements on record-keeping, transparency, human oversight, and accuracy/robustness (Articles 12–15), deployer record obligations (Article 19, Article 26), post-market monitoring (Article 72), and the data-subject-facing explanation right (GDPR Article 22; AI Act Article 86) — are, in large part, **evidence obligations**: they require process artifacts (logs, documentation, assigned oversight, reproducible behaviour), not merely good outputs. A stochastic text generator cannot satisfy an evidence obligation by promising that its output is good; the obligation attaches to *things that exist outside the generator*: records, chains, signatures, named responsibilities.

This paper specifies and verifies such an evidence layer. Our contributions:

- **C1 — A structural language with a machine-checkable grammar.** The Second-Perspective Decision Structure Language (Standard 2026.1): an ISO/IEC 14977 (EBNF) grammar [.spd] with a normative block order, semantic rules R1–R12, 19 diagnostic codes, and a zero-dependency validator/generator. Its design invariant: the language *cannot* express conclusions, recommendations, rankings, scores, or probabilities (enforced as E301–E304); it describes only the boundary within which a decision holds.
- **C2 — A deterministic five-operator audit method.** NS (narrative stripping), IAP (implicit-assumption surfacing), LCH (fragility latch, with a published deterministic ΔD formula), CCS (causal-chain synchronisation with black-hole detection), STATE (responsibility anchoring and verdict aggregation). Explicit severity model: **only structural absence may block; quantitative opinion may not** (Section 4.3).
- **C3 — A reproducibility and tamper-evidence architecture.** A sealed hash chain over audit events with a single published chain root; a model-firewall that makes it impossible for LLM output to influence structural verdicts; and an 11-check independent verification suite any third party can run offline (Section 6). We publish the verified chain root as a cross-machine reference value.
- **C4 — A structural taxonomy.** *Anchored / floating / opaque / hollow* (Section 5): definitions and a fixed measurement procedure, orthogonal to factual truth.
- **C5 — Honest limitations, pre-registered.** Two known gaps are disclosed as first-class results rather than hidden (Section 7): certificate input-binding (V3b) and default-path nonce non-reproducibility (V9).

Scope discipline: the method **does not** decide, rank, recommend, or establish truth. It produces audit conclusions and evidence — nothing else. Every property claimed in this paper is stated with the command or procedure by which a reader can independently check it.

Relation to companion work. Argument papers derived from the same programme — on trust relocation for AI systems, on the categorical distinction between audit and explanation, and on the limits of self-certification — are under review or in preparation. This report is the artifact reference for that programme: it defines the method and specifies its reproducible evidence, and stands on its own.

1.1 Notation and Terminology

Term	Sense in this report	Not
audit	ex-ante reconstruction and verification of the dependency structure of a decision	not post-hoc review of logs or outputs; not inspection of a model's internals
determinism (deterministic)	reproducibility: identical declared inputs yield the identical audit result	not metaphysical determinism; not a claim that the audited process is causally determined
verdict	the audit's terminal classification over declared structure (AUDIT_HALT / AUDIT_WARN / AUDIT_PASS)	not a ruling on the factual truth of the claim; not a legal decision
structure	the declared premises, failure branches and dependencies of a claim	not data structures; not model architecture
anchored (structural state)	a claim whose declared dependencies terminate in committed failure responses — the part no post-hoc narrative can supply	not responsibility anchoring (the STATE operator), which terminates the audit in a named decision unit

2. Background and Related Work

2.1 Governance context

The EU AI Act (Regulation (EU) 2024/1689) imposes, for high-risk systems, a risk-management system (Article 9), technical documentation (Article 11, Annex IV), automatic record-keeping (Article 12), transparency and instructions for use including known limitations and accuracy characteristics (Article 13), effective human oversight (Article 14), and accuracy, robustness and cybersecurity including resilience to manipulation (Article 15). Deployers must retain automatically generated logs (Article 19), assign oversight to competent natural persons, and cooperate with authorities (Article 26); post-market monitoring is required (Article 72). Third-party conformity assessment is the norm for many high-risk categories (Article 43). GDPR Article 22 grants data subjects the right to meaningful information about automated decision logic and to contest decisions.

Complementary frameworks — the NIST AI Risk Management Framework [2], ISO/IEC 42001 [3], and Singapore IMDA's AI Verify testing framework [4] — share a common shape: they require **documented, verifiable process evidence**, maintained over time. This paper's method targets precisely that layer, and was itself assessed against AI Verify (total score 95/100; provider-commissioned report shipped with the artifacts).

2.2 Structural vs. factual defects

Work on hallucination in natural language generation [5] typically defines hallucination as content that is unfaithful to a source or to facts. The stochastic-parrot critique [6] concerns amplification of bias and unfaithfulness from training data. Both lines evaluate **content**. Our taxonomy evaluates **structure**: whether a decision claim declares the assumptions it depends on and the branch responses it commits to. A claim can be factually correct yet *opaque* (assumptions undeclared) — under the regimes above, the second defect is the one that leaves no record when things go wrong.

2.3 Audit independence and reproducibility

Audit as an institution is built on separation [9]: the Sarbanes–Oxley reforms following Enron codified the principle that the auditor and the audited may not be the same party, because self-audit does not constitute evidence. The same architecture appears in AI governance: internal safety teams at model vendors produce advisory evaluations; external evaluators exist precisely because self-assessment does not transfer trust. Reproducibility has an established methodology in machine learning [7]: frozen inputs, disclosed protocols, independently re-runnable artifacts. We adopt both principles: the audit engine is decision-agnostic, vendor-independent, offline, and does not modify or require modification of any audited model; and every claim in this paper is paired with a re-runnable check.

3. The Decision Structure Language (Standard 2026.1)

3.1 Grammar

The language describes exactly four block types, in normative order (Decision < Assumption < Dependency < Branch), plus whole-line comments:

```
document  = decision , assumptions , [ dependencies ] , branches ;
decision  = "Decision" , colon , text ;
assumption = "Assumption" , id , colon , text ;
dependency = "Dependency" , ":" , id , ( "requires" | "depends on" ) , id ;
branch    = "Branch" , id , colon , text ;
```

File extension `.spd`. The formal grammar is provided in ISO/IEC 14977 (EBNF) notation. A `Dependency` states an ordering constraint between two declared assumptions; a `Branch` states the **structural change to the decision** when its assumption fails — it is not an alternative decision.

3.2 Design invariant: non-advisory by construction

The normative standard states the invariant directly: the language *must not* express conclusions, recommendations, rankings, scores, probabilities, or optimisation guidance. The validator enforces this as hard errors:

Code	Meaning
E301	Conclusion-bearing statement
E302	Recommendation statement
E303	Scoring / ranking / probability assignment
E304	Optimisation guidance

A tool that cannot emit advice cannot be recruited into advice-giving — this is a *construction* guarantee, not a behavioural promise.

3.3 Semantic rules and diagnostic codes

Context-free EBNF cannot express the reference integrity of identifiers or graph constraints; those are enforced by the validator as normative rules: every branch id must match a declared assumption id ($R1 \rightarrow E104$); every dependency endpoint must be a declared assumption ($R2 \rightarrow E104$); assumption ids must be unique ($R3 \rightarrow E105$); the dependency graph must be acyclic ($R4 \rightarrow E106$); **every assumption must have exactly one matching branch ($R5 \rightarrow E107$)**; and R6–R9 restate the E301–E304 constraint layer. Three warnings (W401 vague qualifier, W402 missing observable threshold, W403 self-evident assumption) mark structural quality without blocking.

The full diagnostic set is 19 codes: E101–E108 (structural), E201–E204 (lexical), E301–E304 (constraint), W401–W403 (advisory warnings); see Appendix A.

A consequence worth stating precisely: **a document that validates at the E-level already has full branch coverage (R5)**. The "floating" structure of Section 5 — an assumption declared with no failure response — is *representable in prose but not in a valid .spd record*. The language makes the defect unrepresentable at validation time; the audit pipeline (Section 4) is what detects it in prose.

3.4 Toolchain determinism

The validator and generator (`dsl.py`) are zero-dependency (Python standard library only), deterministic, and seeded: generation with `--seed 2026 --count 5` produces byte-identical output across runs (verified, Section 6, V7). The validator ships a bilingual (EN/ZH) constraint lexicon; prose defaults to English while Chinese records are fully checked. Valid records exit 0; invalid records exit 1 — CI-usable.

4. The Audit Method (GCAE v2.0.0)

4.1 Input contract

The engine audits a *decision context*: a dictionary whose keys are all optional, with documented aliases:

Key	Aliases	Role
narrative	background, summary, description	decision statement possibly containing rhetoric; NS strips it
decision	p, premise, action	the P of the causal chain
assumptions	premises, hypotheses	the A — declared premises
outcome	q, result, consequence	the Q
branches	branch_responses, failure_paths, delta_d	[{"assumption": "A1", "delta_d": "..."}]; semantics $\neg A \Rightarrow \Delta D$
dependencies	dependency_graph, deps	{"A1": ["A2"]}
criteria	—	{dimension: {"weight": w}}; weights must sum to 1.0
evidence	—	identifiers supporting the conclusion

Minimal usable input: `decision`, `assumptions`, `outcome`. Free prose can be audited by placing it in `narrative`. Absence of keys is never an error — it is itself an auditable condition (missing assumptions are *reported*, not *guessed*).

4.2 The five operators

NS — Narrative Stripping. A deterministic lexicon pass over four marker classes — emotional intensifiers, moral/stance padding, vague quantifiers, authority appeals — plus regex patterns for vague numeric quantifiers. Output: the *logical core* (text with markers excised) and the violation list. Zero randomness, zero LLM calls.

IAP — Implicit Assumption Surfacing. Detects: self-referential premises; **privilege bypass** (e.g. "no approval required", 特批, exemption from review — severity **HALT**); unilateral premises (one-sided conditions); circular justification ($P \Rightarrow Q$, severity **HALT**); and missing assumptions (a decision declared with no assumptions — severity **WARN**). The engine additionally reports *conclusion without evidence* and *non-normalised criteria weights* when those inputs are present and defective.

LCH — Fragility Latch. For each declared assumption A_i , computes a deterministic collapse probability ΔD :

```

 $\Delta D(A_i)$  = clamp[0,1]( 0.30           # base value
    + 0.30  if no branch response      #  $\neg A_i$  has no fallback
    - 0.10  if branch response         # the fallback exists
    + 0.15  $\times$  N                      # N = number of dependents
    + 0.10  $\times$  M                      # M = vague qualifiers
    + 0.25  if non-falsifiable )      # "always/never" — no detector signal

```

Weight rationale (published as design constants, **not statistically fitted**): the most common failure combination — no branch response + non-falsifiable — sums to $0.85 > 0.7$, failing; with a branch response it is $0.45 < 0.7$, passing. The threshold 0.7 therefore separates one thing: *whether a fallback path was prepared for this assumption*. `system_delta_d` is the maximum over assumptions; pass requires `system_delta_d < 0.7` **and** full branch coverage.

CCS — Causal Chain Synchronisation. Four checks: (1) *inverse check* — if $\neg P$ holds, is the system fallback-convergent or collapse-prone (no branches $\rightarrow$ **HALT**, **SYSTEM_COLLAPSE**); (2) *counterfactual check* — is $\neg P \Rightarrow Q'$ considered; (3) *chain integrity* — are P, A, Q all present (structural, not semantic, coherence); (4) *black-hole detection* — any missing key variable (P, A, or Q) is a **HALT**: **the chain breaks; no speculative completion is performed**. Documented limitation: the inverse check counts branch *quantity*, not branch–assumption *identity* correspondence; the identity-matched check lives in LCH. The two checks use different coverage definitions, and this is disclosed in the module header (Section 7, L7).

STATE — State Anchoring. Anchors responsibility to a minimal decision unit: `ResponsibilityAccount(organization, role, stage, owner, nonce)`. Vague organisations ("team", "company", "everyone") are flagged **UNANCHORED**. STATE aggregates NS/IAP/LCH/CCS into a verdict — `AUDIT_HALT` if any halt exists, else `AUDIT_WARN` if any warning, else `AUDIT_PASS` — and generates the audit certificate: `audit_id = SPL-{nonce}-{timestamp}` with `signature = SHA-256(org | role | stage | nonce | verdict-level | halt_count | warn_count | timestamp)`.

4.3 Severity model: only structural absence may block

Plugins carry a permission tier. Structural operators (CCS: missing key variables; STATE: final adjudication) may emit HALT and block an audit. Signal operators (IAP, LCH) may emit risk signals but cannot block. Narrative operators (NS) emit text only; any status field in their output is stripped. The design rule, verbatim from the implementation: 只有「结构性缺失」才能阻断，「量化意见」不能 — structural absence blocks; quantitative opinion does not. This is what keeps the method falsifiable in the right way: it never condemns a decision for being *unlikely*; it condemns only declared structure that is *absent*.

4.4 The model firewall

The engine's optional LLM narrative adapter (off by default) is permission-tiered, and a fixed key set — `status`, `converged`, `blocked`, `verdict`, `decision`, `weight`, `score`, `rank`, `adjudicated`, `is_converged` — is **stripped from any LLM output** before it can touch structural adjudication. Even if a model is prompt-injected into returning verdict fields, they cannot enter the audit result. The audit layer is structurally independent of the audited model: no model modification is required or possible, and model output cannot influence verdicts.

4.5 Evidence chain and reconstruction

Every audit step appends an event — `AuditEvent(event_type, payload, prev_hash, timestamp, nonce)` — to a hash chain. Each event's nonce is derived deterministically (`SHA-256(prev_hash|event_type|index)[:8]`), and its hash is **sealed at append time**: from that moment, any modification of payload, type, or linkage fails verification. The **chain root** — the last sealed hash — is the single audit fingerprint, and the documentation requires it to be stored separately from the report itself: *a root stored next to its report is no anchor at all*.

The engine also supports causal *reconstruction*: injecting corrected delta-variables into a decision context and testing convergence, classified into five states — `FIXED_POINT` and `NO_GAIN` are true convergence; `BUDGET_EXHAUSTED` is explicitly *not* (running out of rounds is not the same as settling); `DIVERGED` and `BLOCKED` halt for human intervention.

5. A Structural Taxonomy of Decision Claims

We define four structural states for a decision claim. The taxonomy is independent of factual truth — it asks only what structure the claim declares.

State	Definition	Scoring
anchored	Premise assumptions and their failure branches are both declared; structure closes (every critical assumption has a committed response).	pass
floating	Assumptions declared, but the failure response ($\neg A \Rightarrow \Delta D$) is missing.	pass (through)
opaque	No assumptions declared.	fail
hollow	No assumptions declared and no substantive content (a lexical black hole).	fail

Measurement mapping to the pipeline: `hollow` $\approx$ CCS black-hole (missing P/A/Q) with an empty NS logical core; `opaque` $\approx$ IAP `missing_assumptions` on a decision bearing a conclusion; `floating` $\approx$ LCH branch-coverage failure / CCS inverse `SYSTEM_COLLAPSE`; `anchored` $\approx$ verdict not HALT with full branch coverage. Because the language of Section 3 makes full branch coverage a validation requirement (R5), a *valid .spd record is anchored by construction* — the taxonomy's role is to grade **prose** claims, and to give the audit a fixed, pre-registerable measurement procedure (thresholds must be fixed before data collection; e.g. a batch with hollow-rate > 20% triggers a HALT of the batch).

6. Properties and Verification

Every property below is paired with the check that verifies it. The verification suite (`verify.py`) is zero-dependency, offline, and CI-friendly (exit code 0 = no FAIL). **Results of a full run on 2026-10-09: 11 checks — PASS 9 · FAIL 0 · WARN 2; exit code 0.**

#	Property	Statement	Verified by
P1	Offline zero-dependency	Engine and all five operators load using the Python standard library only; no network access.	V1 (PASS)
P2	Content determinism	Identical frozen input + clock + nonce $\Rightarrow$ identical chain root across runs.	V2: 3 consecutive runs, identical root (PASS)
P3	Input sensitivity	Changing audited input changes the chain root (the hash cannot degenerate into a constant).	V3: two forced mutations, both roots changed (PASS)
P4	Chain integrity	An untampered chain verifies.	V4 (PASS)
P5	Tamper evidence	Mutating event payload, event type, or linkage is detected.	V5: all three mutations detected (PASS)
P6	Blocking on unenforceable responsibility	Missing responsible owner $\Rightarrow$ <code>RESPONSIBILITY_CLOSURE = BLOCKED</code> ; once closed, no longer blocks.	V6 (PASS)
P7	Language-toolchain determinism	Seeded generation is byte-identical across runs; valid/invalid samples exit 0/1.	V7, V7b (PASS)
P8	Dependency hygiene	Shipped dependency manifest contains no unrelated GUI/packaging deps for the audit core.	V8 (PASS)
P9	Reproducible reference root	The chain root for the frozen reference context, comparable across machines: <code>f5713f49481a80f82356ba1255c0292ddb0d17a235a35348bcff0eef49d6a801</code>	<code>python verify.py --root</code>
L1	Certificate coverage limited to derived report	The chain root hashes the derived audit report, not the raw input; a certificate cannot independently prove <i>which input</i> was audited. Disclosed trade-off: sensitive decision data is not persisted.	V3b (WARN, disclosed)
L2	Default-path nonce non-reproducibility	Default nonce is <code>uuid4</code> ; byte-level reproduction requires explicitly fixing clock and nonce (all shipped scripts do).	V9 (WARN, disclosed)

Three further reproduction facts: (i) the end-to-end demonstration halts a decision that is rhetorically padded, assumption-hidden, and branch-less, while passing a fully-structured decision; (ii) the engine gates on stage whitelists at construction (invalid responsibility node $\Rightarrow$ no audit at all); (iii) the chain root is the *only* external verification credential, and the suite's `--root` mode prints it as a single line for cross-machine comparison.

Claim type discipline. P1–P8 are categorical claims: each is a binary fact, and a single counterexample would refute it — its statement includes the exact procedure producing the counterexample-attempt. P9 is a reproducibility artifact. The paper deliberately makes **no statistical superiority claim** about any other system; such claims would be comparative, not categorical, and are left to per-deployment evaluation under a pre-registered protocol (Section 8).

7. Limitations

Disclosed as results, not footnotes:

- **L1 (V3b — certificate input-binding).** The certificate covers the derived report, not the raw input. Rationale: privacy by design (sensitive decision data is not persisted). Fix if input-binding is required: fold a normalised digest of the input into

the hashed report. Until then, certificate coverage must be stated explicitly to counterparties — never assumed.

- **L2 (V9 — nonce).** Default-path audits are not byte-reproducible. Reproduction requires fixing clock and nonce, as every shipped script does. Fix direction: derive the nonce deterministically from account fields.
- **L3 (LCH weights).** ΔD constants are deterministic design values, **not statistically fitted**. Changing them changes verdicts; that is a design change, not parameter tuning. The threshold's discriminating behaviour is transparent (§4.2) and falsifiable.
- **L4 (CCS inverse coverage).** Counts branch quantity, not identity-matched correspondence (LCH does the identity-matched check). Two coverage definitions coexist by design decision and are documented in the module header.
- **L5 (Lexicons are literal).** NS/IAP match lexical patterns; they do not understand semantics. A claim can evade detection through paraphrase. This bounds the method's recall; it is a detector of declared structure, not an interpreter of meaning.
- **L6 (No open-domain performance claim).** No pre-baked question bank ships with the artifacts: synthetic corpora have no discriminative power for real deployment targets. Evaluation must use the deployer's own real objects and corpus, under a pre-registered protocol (frozen thresholds, paired arms, zero-flip re-runs, Wilson intervals, exact McNemar, dual annotation with $\kappa \geq 0.70$).
- **L7 (Self-assessment context).** The AI Verify score (95/100) comes from a provider-commissioned assessment of the artifacts, not from the present document. It is a precedent, not an independent audit of this paper's claims. The claims of this paper stand on their own reproduction procedures (§6).

The method makes **no factual-truth judgement**, no probability judgement about outcomes, and no recommendation. It audits declared structure and produces evidence; nothing more is claimed.

8. Discussion: Why an Audit Layer Cannot Be Merged Into the Generator

The properties above (P2–P6, P9) are not features of any particular vendor's model; they are *category properties of an evidence layer*:

1. **Stochastic outputs cannot supply reproducibility.** Content determinism (P2) conflicts with sampled generation by construction. A generator can promise stable behaviour; it cannot emit byte-reproducible evidence without ceasing to be the generator.
2. **Self-audit is not audit.** Record-keeping, oversight, and conformity assessment regimes are built on separation between the assessing and the assessed party [9]. An audit unit owned by the audited is, institutionally, an internal tool — its findings are advisory to the same incentive structure. The AI Act's own architecture (Articles 14, 26, 43, 72) presumes this separation.
3. **Vendor-neutrality is relational.** A multi-model enterprise operation requires one ruler across models. An audit layer bound to a single vendor is structurally disqualified from comparing across vendors; a vendor-free layer is the only architecture that scales across a heterogeneous model market.
4. **Evidence outlives capability.** Model versions are silently updated; certificates, chain roots, and standards language versions are versioned and frozen. When oversight or litigation asks "what did the system commit to, on what assumptions, under whose signature", only the frozen record answers.

The complementary claim follows: generators **cover**; the audit layer **vouches**. The method does not attempt to beat any model at understanding — it supplies what no model output can carry: declared structure, reproducible evidence, anchored responsibility.

9. Conclusion

We specified a deterministic, offline, model-agnostic method for auditing the *structure* of decision claims: a non-advisory formal language (19 diagnostic codes; conclusions/recommendations/rankings/optimisation rejected by construction), a five-operator pipeline with a severity model in which only structural absence may block, a structural taxonomy (anchored/floating/opaque/hollow) orthogonal to factual truth, and a sealed hash-chain evidence architecture with a published

verification suite (PASS 9 · FAIL 0 · WARN 2) and a cross-machine reference chain root. Two limitations are shipped as first-class disclosures. This report is the first public, self-contained description of the method and its reference artifacts; its claims are categorical and each is paired with its own refutation procedure. What it establishes is narrow and durable: **the record that a decision declared its assumptions and prepared its failure branches — or did not**. That record, not the narrative, is what survives an audit.

References

1. Regulation (EU) 2024/1689 of the European Parliament and of the Council (Artificial Intelligence Act), *Official Journal of the European Union*, 2024.
2. National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, January 2023.
3. ISO/IEC 42001:2023, *Information technology — Artificial intelligence — Management system*.
4. AI Verify Foundation (Infocomm Media Development Authority, Singapore), *AI Verify: An AI Governance Testing Framework and Toolkit*.
5. Z. Ji, N. Lee, R. Frieske, et al., "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, 55(12):248, 2023.
6. E. M. Bender, T. Gebru, A. McMillan-Major, S. Shmitchell, "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?," *Proc. ACM FAccT*, 2021.
7. J. Pineau, P. Vincent-Lamarre, K. Sinha, et al., "Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program)," *Journal of Machine Learning Research*, 22(132):1–20, 2021.
8. C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, 1:206–215, 2019.
9. M. Power, *The Audit Society: Rituals of Verification*. Oxford University Press, 1997.
10. R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," *Advances in Cryptology — CRYPTO '87*, 1987.

Artifacts. Second-Perspective repository (GCAE v2.0.0; language Standard 2026.1; verification suite): <https://github.com/nohn3043-arch/second-perspective>. Artifacts cited in this paper: `Cognitive Audit Engine.py`; `plugins/{ns,iap,lch,ccs,state}.py`; `language Standard/{2026.md,decision.ebnf,dsl.py}`; `demo_audit.py`; `verify.py`; `TESTING.md`. See `LICENSE` for terms (personal non-commercial research free; government/enterprise use requires a commercial license).

Appendix A — The 19 Diagnostic Codes (Language Standard 2026.1)

Code	Level	Meaning
E101	ERROR	Missing Decision block
E102	ERROR	Duplicate Decision block
E103	ERROR	Missing Assumption block
E104	ERROR	Reference to an undeclared assumption id
E105	ERROR	Duplicate id declaration
E106	ERROR	Cycle in the dependency graph
E107	ERROR	Assumption without a matching Branch
E108	ERROR	Block order violated
E201	ERROR	Unrecognised line (unknown keyword)
E202	ERROR	Invalid identifier format
E203	ERROR	Empty text
E204	ERROR	Missing colon or separator
E301	ERROR	Conclusion-bearing statement
E302	ERROR	Recommendation statement
E303	ERROR	Scoring / ranking / probability assignment
E304	ERROR	Optimisation guidance
W401	WARNING	Assumption contains a vague qualifier
W402	WARNING	Assumption lacks an observable threshold
W403	WARNING	Assumption looks self-evident

Appendix B — ΔD Worked Example (from the fragility module)

Assumption: "需求大概稳定" (*demand is probably stable*) with no branch response, no dependents, one vague qualifier, falsifiable:

$$\Delta D = 0.30 \text{ (base)} + 0.30 \text{ (no branch)} + 0.10 \times 1 \text{ (vague)} = 0.70 \rightarrow \text{fails threshold}$$

Same assumption with a branch response: $0.30 - 0.10 + 0.10 = 0.30 \rightarrow$ passes. A non-falsifiable variant with no branch response ("需求永远稳定"): $0.30 + 0.30 + 0.25 = 0.85 \rightarrow$ fails; the same variant *with* a fallback drops to $0.30 - 0.10 + 0.25 = 0.45 \rightarrow$ passes. The branch response is the single largest lever: a prepared fallback absorbs even an unfalsifiable premise, whereas an unfalsifiable premise without a fallback is the worst case — its failure is discovered the moment the decision collapses.

Appendix C — Reproduction Checklist (any third party, offline)

```
git clone https://github.com/nohn3043-arch/second-perspective
cd second-perspective
python verify.py          # expect: PASS 9 · FAIL 0 · WARN 2; exit code 0
python verify.py --root   # expect: f5713f49481a80f82356ba1255c0292ddb0d17a235a35348bcff0eef49d6a801
python demo_audit.py      # expect: dirty decision HALT; clean decision PASS/WARN
python "language Standard/dsl.py" check "language Standard/examples/valid_decision.spd" # exit 0
python "language Standard/dsl.py" check "language Standard/examples/invalid_decision.spd" # exit 1
python "language Standard/dsl.py" gen --seed 2026 --count 5 --out samples/ --self-check # byte-reproducible
```